Chapter 13 Red-Black Trees

1) A binary search tree of height h can implement any of the basic dynamic-set operations in time O(h). The set operations are fast if the height of the search tree is small. But if its height is large their performance may be no better than with a linked list.

Red black trees are one of many search-tree schemes that are balanced in order to guarantee that basic dynamic-set operations take O(lgn) time in worst case.

2) Properties of red-black trees :-

A red-black tree is a binary search tree with one extra bit of storage per node: its color, which can be either RED or BLACK. By constraining the way nodes can be colored on any path from the root to a leaf, red-black trees ensure that no such path is more than twice as long as any other, so that the tree is approximately balanced.

A binary search tree is a red-black tree if it satisfies the following red-black properties:-

1) Every node is either red or black.
2) The root is black.
3) Every leaf (NIL) is black.
4) If a node is red, then both its children are black.
5) For each node, all paths from the node to "descendant leaves" contain the same no. of black nodes.

→ Every node of red-black tree contains the fields color, key, left, right and parent. If a child or the parent of a node does not exist, the corresponding pointer field of the node contains the value NIL. These NIL's are pointers to external nodes (or leaves) of the binary search tree and the normal, key-bearing nodes are the internal nodes of the tree.

We use a sentinel value sril[T] to represent NIL. sril[T] is an object with the same fields as an ordinary node in the tree. Its color field is black and its other fields can be set to arbitrary values. We confine our interest to the internal nodes of a red black tree, since they hold the key values.

→ Black height of a node:

Black height of a node x is defined as the number of black nodes on every path from node x, that does not include x, down to a leaf. It is denoted by $bh(x)$.

The no. of black nodes on every path from x (excluding x) down to some descendant leaf would be the same (from red-black property).

The black-height of a red-black tree is the black-height of its root.

3) Lemma

A red-black tree with n internal nodes has a maximum height of $2 \lg(n+1)$

Proof

We start by showing that the subtree rooted at x contains at least $2^{bh(x)} - 1$ internal nodes.

We prove this claim by induction on the height of x. If the height of x is 0, then x must be a leaf, and the subtree rooted at x indeed contains at least $2^{bh(x)} - 1 = 2^0 - 1 = 0$ "internal nodes".

For the inductive step, consider an internal node with 2 children. The children could be either red or black. If a child is red its black-height is $= bh(x)$. If a child is black, its black-height is $bh(x)-1$ as that child has now been excluded. Thus each child has at least $2^{bh(x)-1} - 1$ internal nodes. Thus the subtree rooted at x contains at least $2 \cdot (2^{bh(x)-1} - 1) + 1$

$= 2^{bh(x)} - 1$ internal nodes which proves the claim.

To complete the proof of the lemma, let the height of the tree be h. Consider the longest path from the root to any leaf. Whenever you encounter a red node it must have a black child on this path. Thus at least half the nodes on any simple path from the root to a leaf, excluding the root, must be black. Hence, the black height of the root must be at least $h/2$. Thus :-

$$n \geq 2^{h/2} - 1 \Rightarrow n + 1 \geq 2^{h/2}$$

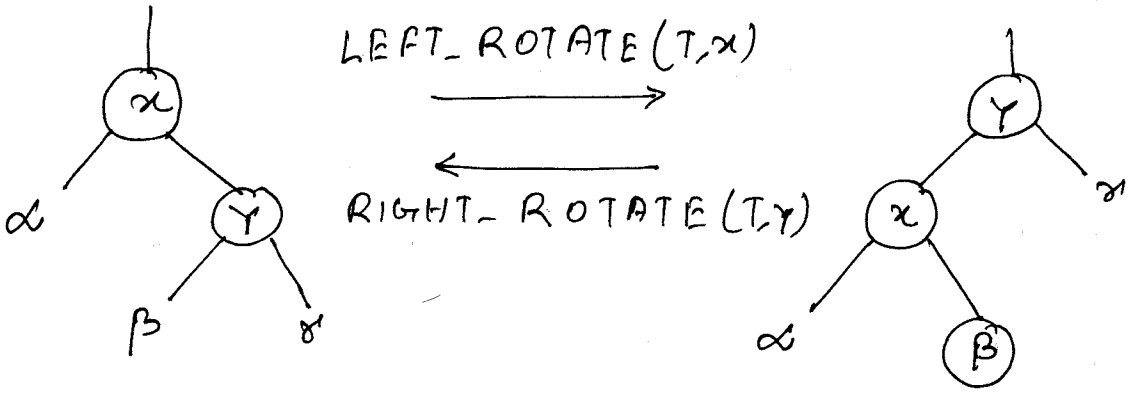
$$\therefore h \leq 2 \lg(n+1)$$

→ Since a red-black tree on n nodes is a binary search tree with height $O(\lg n)$, all of the basic querying operations on dynamic-set (such as SEARCH, PREDECESSOR, SUCCESSOR, MINIMUM, MAXIMUM) can be made to run in $O(h) = O(\lg n)$ time.

4) Rotations:

We change the pointer structure of a binary search tree through rotation. Rotation is a local operation that preserves the binary search tree property.

2 kinds of rotations are possible:-



→ x, y are nodes of the subtree T

→ α, β, γ represent arbitrary subtrees

→ A rotation operation preserves the binary search tree property: the keys in α precede key[x], which precedes the keys in β, which precede key[y] which precedes the keys in γ.

→ When we do a left rotation on a node x, we assume that its right child is not nil[T]. Left rotation pivots around the link from x to y. It makes y the

new root of the subtree, with x as the left-child of y and y's left child as x's right child.

→ The pseudocode for LEFT_ROTATE(T, x) assumes that right[x] ≠ nil[T] and that the root's parent is nil[T].

LEFT_ROTATE(T, x)

```
1   y ← right[x]            ▷ initialize y
2   right[x] ← left[y]      ▷ turn y's left subtree
                               into x's right subtree
3   parent[left[y]] ← x
4   parent[y] ← parent[x]
5   If parent[x] = nil[T]
        then root[T] ← y
6       else if x = left[parent[x]]
7               then left[parent[x]] ← y
8               else right[parent[x]] ← y
9   left[y] ← x
10  parent[x] ← y           ▷ Note that to change
```

a single link you need to change 2 pointers

→ LEFT_ROTATE and RIGHT_ROTATE run in O(1) time.

5) Insertion

Insertion of a node into a red-black tree is done using a procedure called RB_INSERT (T,z). RB-INSERT first inserts the node z into the tree T as if it were an ordinary binary search tree. Then it colors the node z red. Coloring z red may cause a violation of the red-black properties. Hence, RB_INSERT next invokes RB_INSERT_FIXUP to recolor nodes and perform rotations, to restore the red-black properties.

RB_INSERT (T,z)

```
1   trav ← root[T]
2   parent_trav ← NIL[T]
3   While trav ≠ NIL[T]
4         do parent_trav ← trav
5            if Key(z) < Key[trav]
6               then trav ← left[trav]
7               else trav ← right[trav]
8   parent[z] ← parent_trav
9   if parent_trav = NIL[T]
10        then root[T] ← z
```

```
11      else if  key[z] < key[parent-trav]
12               then  left[parent-trav] ← z
13               else  right[parent-trav] ← z
14      left[z] ← NIL[T]
15      right[z] ← NIL[T]
16      color[z] ← RED
17      RB-INSERT-FIXUP(T, z)
```

→ There are 4 differences between the procedures TREE-INSERT and RB-INSERT:

(i) all instances of NIL have been replaced by NIL[T]

(ii) we set left[z] and right[z] to NIL[T] in order to maintain proper tree structure

(iii) we color z red

(iv) we invoke RB-INSERT-FIXUP to restore the red-black properties

→ **Which of the red-black properties can be violated upon the call to RB-INSERT-FIXUP?**

Property 1 continues to hold as every node is either red or black. Property 3 continues to hold as the newly created leaves are the sentiel NIL[T] which are black. The red node added to the binary search tree replaced a sentinel NIL[T] (a black node) &

but created 2 sentinels (NIL[T]) as it's children which now exist on different downward simple path from the root. Thus the no. of black nodes on every simple downward path from a node to it's leaves remains the same. This proves that property 5 continues to hold.

If the new node was inserted into a previously empty red-black tree it becomes the root of it. Since, the root is now red, property 2 is violated.

If the newly inserted node becomes the child of a red node, property 4 is violated.

Thus, the only red black properties that can be violated are property 2 and property 4. If there is a violation of red-black properties, there is at most one violation and it is either of property 2 or property 4.

→ If property 2 is violated we restore property 2 by executing color[root T] ← black.

→ If property 4 is violated we actually have six cases which are divided into 2 sets of 3 cases each. The 2 sets are symmetric and depend upon whether Z's parent is a left child or a right child of Z's grandparent. (If property 4 is the only

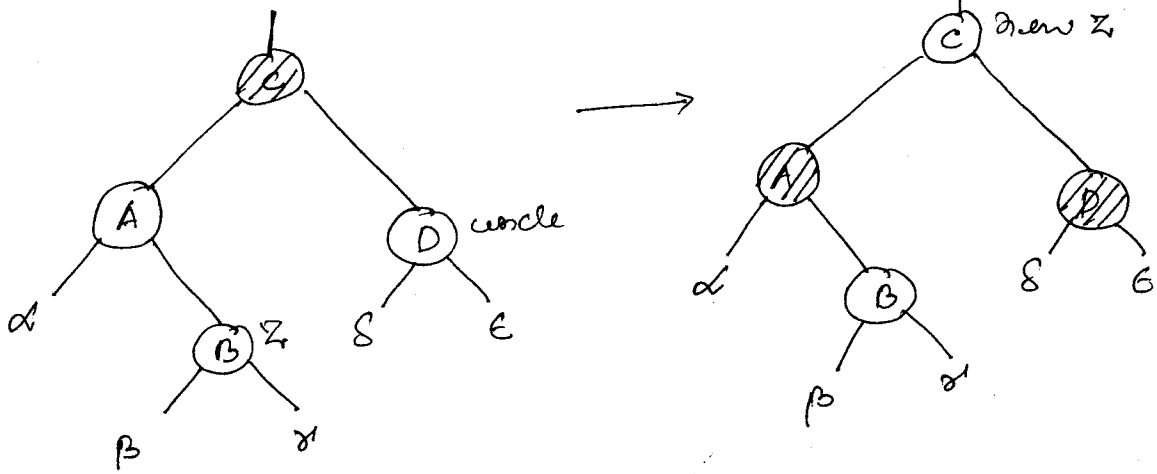
violated property, z should have both a parent and a grandparent. z certainly has a parent which is red. Since parent of z is red, it is not the root (property 2 is not violated so root[T] must be black). Hence, z certainly has a grandparent, which may or may not be the root.

The three cases that arise are :-

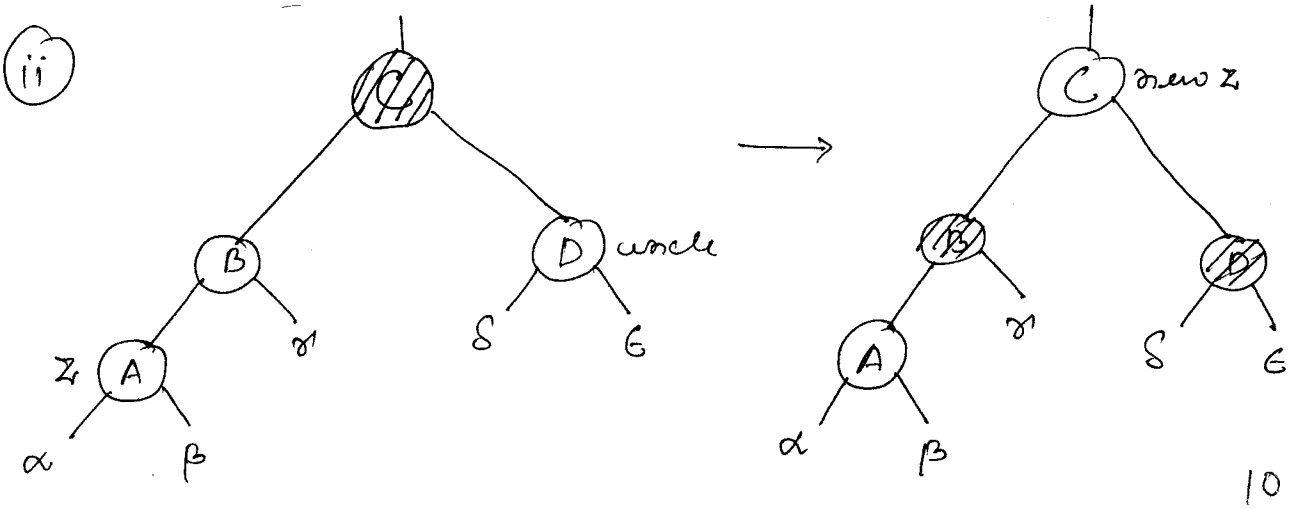
1) Case 1 : z's uncle is red

2 subcases arise. z may be the right child or the left child of its parent.

(i)



(ii)



Case 1 sets the colors of the parent and the uncle of z to black.

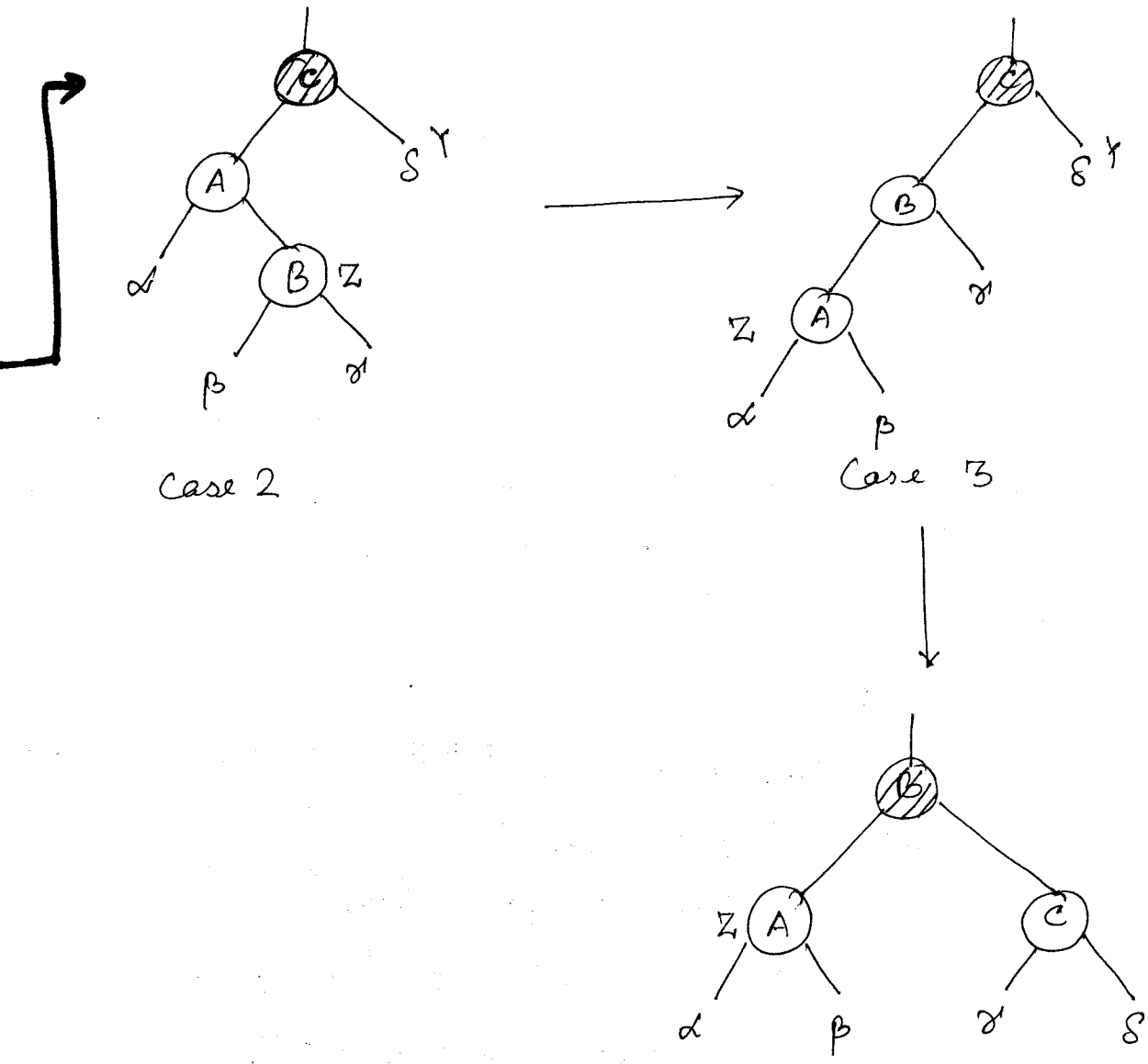
2) **Case 2: z's uncle is black and z is a right child**

Case 3: z's uncle is black and z is a left child

The grandparent of z was previously black (as property 4 was only violated between nodes z and parent of z). The grandparent of z is now colored red and z is now set to point to it's grandparent.

Any violation of property 4 can now occur only between the new z, which is red, and it's parent if it is red as well.

Note that case 1 only recolors the nodes in such a fashion that preserves property 5. The change in color of the grandparent from black to red is compensated by the change in colors of parent and uncle from red to black. All downward paths from a node to a leaf still have the same no. of blacks.



Case 2 :-

(i) first, makes Z point to its parent and then,

(ii) performs left rotation about Z to transform case 2 to case 3.

First promoting 3 and then left-rotating about 3, makes 3 the left child of its original parent.

Case 3 :-

(i) colors the parent of 3 black
(ii) colors the grandparent of 3 red, and then
(iii) performs a right-rotation about the grandparent of 3.

Since, we no longer have two red nodes in a row property 4 can no longer be violated. Cases 2 and 3 correct the lone violation of property 4 and they do not introduce another violation.

RB-INSERT-FIXUP(T,Z)

```
1  while color[parent[z]] = RED          ▷ while property
                                         ▷ 4 is being
2      do if parent[z] =                 ▷ violated
           left[parent[parent[z]]]
3        then uncle ← right[p[p[z]]]
4             if color[uncle] = RED
5               then color[p[z]] ← BLACK
                                         ▷ Case 1
6                    color[uncle] ← BLACK
7                    color[p[p[z]]] ← RED
8                    z ← p[p[z]]
                                         ▷ End of case 1
```

```
9        else if z = right[p[z]]
10           then z ← p[z]                    ▷ Case 2
11                LEFT-ROTATE(T,z)            ▷ Case 2
12           color[p[z]] ← BLACK              ▷ Case 3
13           color[p[p[z]]] ← RED             ▷ Case 3
14           RIGHT-ROTATE(T,p[p[z]])          ▷ Case 3
15       else (same as then clause with
              "right" and "left" exchanged)
     ▷ the symmetric set of cases
16   color[root[T]] ← BLACK
```

→ What is the running time of RB-INSERT?

All the lines of RB-INSERT, except the call to RB-INSERT-FIXUP takes $O(\lg n)$ time.

In RB-INSERT-FIXUP if property 4 is violated, the while loop runs a single time as case 2 or case 3 fixes up property 4 in a single run. Thus, the while loop repeats only if case 1 is executed, and the pointer moves 2 levels up the tree. The total no. of times the while loop can be executed is therefore $O(\lg n)$. Thus RB-INSERT takes a total of $O(\lg n)$ time. [Moreover, it never performs more than 2 rotations as the while loop terminates

if case 2 or case 3 is executed.)

6) DELETION

The procedure RB_DELETE first splices out a node considering red-black tree as an ordinary binary search tree. Next, it invokes RB_DELETE_FIXUP to restore the red-black properties.

RB-DELETE (T,z)

```
1    If left[z] = NIL[T] or right[z] = NIL[T]
2       then splice-node ← z
3       else splice-node ← TREE_SUCCESSOR(z)
4    if left[splice-node] ≠ NIL[T]
5       then splice-child ← left[splice-node]
6       else splice-child ← right[splice-node]
7    parent[splice-child] ← parent[splice-node]
8    if parent[splice-node] = nil[T]
9       then root[T] ← splice-child
10      else if splice-node = left[parent[splice-
                                   node]]
11                 then left[parent[splice-
                              node]] ← splice-child
12                 else right[parent[splice-node]]
```

← splice-child

13 if splice-node ≠ z

14 then key[z] ← key[splice-node]

15 copy splice-node's satellite data onto z

16 if color[splice-node] = BLACK

17 then RB-DELETE-FIXUP (T, splice-child)

18 return splice-node

→ There are three differences between TREE-DELETE and RB-DELETE :-

(i) all instances of NIL have been replaced by the sentinel NIL[T] in RB-DELETE.

(ii) we no longer perform the check whether splice-child is nil or not. If the splice-child is NIL[T] even then it's parent pointer is set to the parent of the spliced out node.

(iii) if the spliced out node is black, a call to RB-DELETE-FIXUP is made to restore the red-black properties.

→ If the spliced out node is red, the red-black properties still hold for the following reasons:-

(i) if the spliced-out node was red, it could not have been the root. The root continues to be black. Hence, property 2 continues to hold.

(ii) Neither the parent, nor the child of the removed red node must have been red. Hence, no red node have been made adjacent. Property 4 continues to hold.

(iii) Removal of a red node cannot change the black counts along any of the paths. Hence, property 5 continues to hold.

→ The node splice-child passed to RB-DELETE-FIXUP is one of 2 nodes:-

(i) either the node which was splice-node's sole child before splicing, or

(ii) if splice-node had no children, the sentinel NIL[T]

→ Which of the red-black properties can get violated upon the call to RB-DELETE-FIXUP?

Actually properties 2, 4 and 5 can get violated. But we modify the situation in such a manner that property 5 continues to hold and let property 1 stay dissatisfied.

(i) If the splice-node had been the root and the red-child of splice-node becomes the new root, we have a violation of property 2.

(ii) If both splice-child and parent of splice-node were red, after splicing, we have 2 red nodes in a row. Thus, we have a violation of property 4.

(iii) Since this splice-node is a black node, removal of splice-node ~~caus~~ causes any path that previously contained splice-node to have one fewer black node. Thus, property 5 is now violated by any "ancestor" of splice-node in the tree.

We can rectify this problem by saying that splice-child has an extra black. When we splice out the black node we push its blackness onto its child. Property 5 now continues to hold because the count of black nodes on any path that contains splice-child remains the same.

Property 1 now stands violated because splice-child is neither red nor black. Instead, splice-child is either "doubly-black" or "red-and-black" and it contributes either 2 or 1, respectively, to the count of black nodes on paths containing it. The color attribute of

splice-child will be either RED (if splice-child is red-and-black) or black (if splice-child is doubly-black).

→ **RB-DELETE-FIXUP(T,x)**

```
1   while x ≠ root[T] and color[x] = BLACK
2       do if x = left[parent[x]]
3           then sibling ← right[parent[x]]
4           if color[sibling] = red
5               then color[sibling] ← black  ▷Case1
6                   color[parent[x]] ← RED
7                   LEFT_ROTATE(T, parent[x])
8                   sibling ← right[parent[x]] ▷Case1
9           if color[left[sibling]] = BLACK and
               color[right[sibling]] = BLACK
10              then color[sibling] ← RED   ▷C2
11                  x ← parent[x]           ▷C2
12              else if color[right[sibling]]
                    = BLACK
13                  then color[left[sibling]]
                        ← BLACK             ▷C3
14                  color[sibling] ← RED ▷C3
```

```
15          RIGHT_ROTATE ( T, sibling)          ▷C3
16          sibling ← right[parent[x]]          ▷C3
17      color[sibling] ← color[parent[x]]       ▷C4
18      color[parent[x]] ← BLACK                ▷C4
19      color[right[sibling]] ← BLACK           ▷C4
20      LEFT_ROTATE (T, parent[x])              ▷C4
21      x ← root[T]                             ▷C4
22  else (same as then clause with "right"
      and "left" exchanged)
23  color[x] ← BLACK
```

→ The goal of the while loop is to move the extra black up the tree until :-
(any node pointed to by x has an extra black)

(1) x points to a red-and-black node in which case we color x (singly) black in line 23

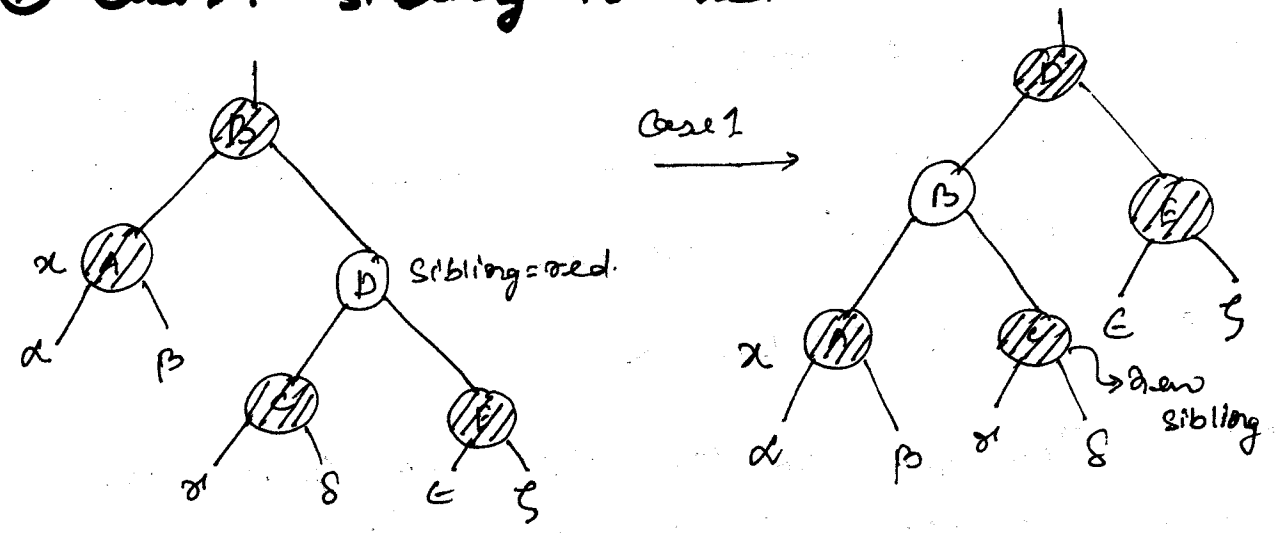
(2) x points to the root in which case the extra black can be simply removed, or

(3) suitable rotations and recolorings can be performed

Within the while loop x always points to a non-root doubly black node.
2 sets of symmetric cases arise when x is a left-child of its parent and when x is a right-child of its parent.
Sibling of x cannot be nil[T].

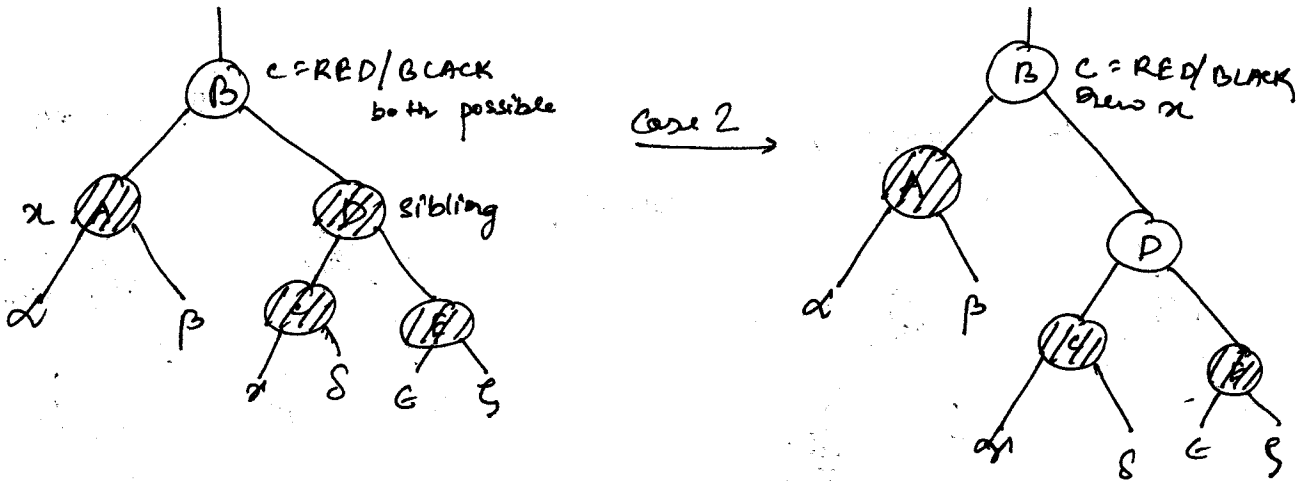
4 cases arise :-

(i) Case 1: Sibling is red



Case 1 transforms itself into case 2 or 3 or 4 by switching the colors of parent and sibling of x and then performing a left-rotation on parent[x].

(ii) Case 2 : sibling is black, and both of sibling's children are black

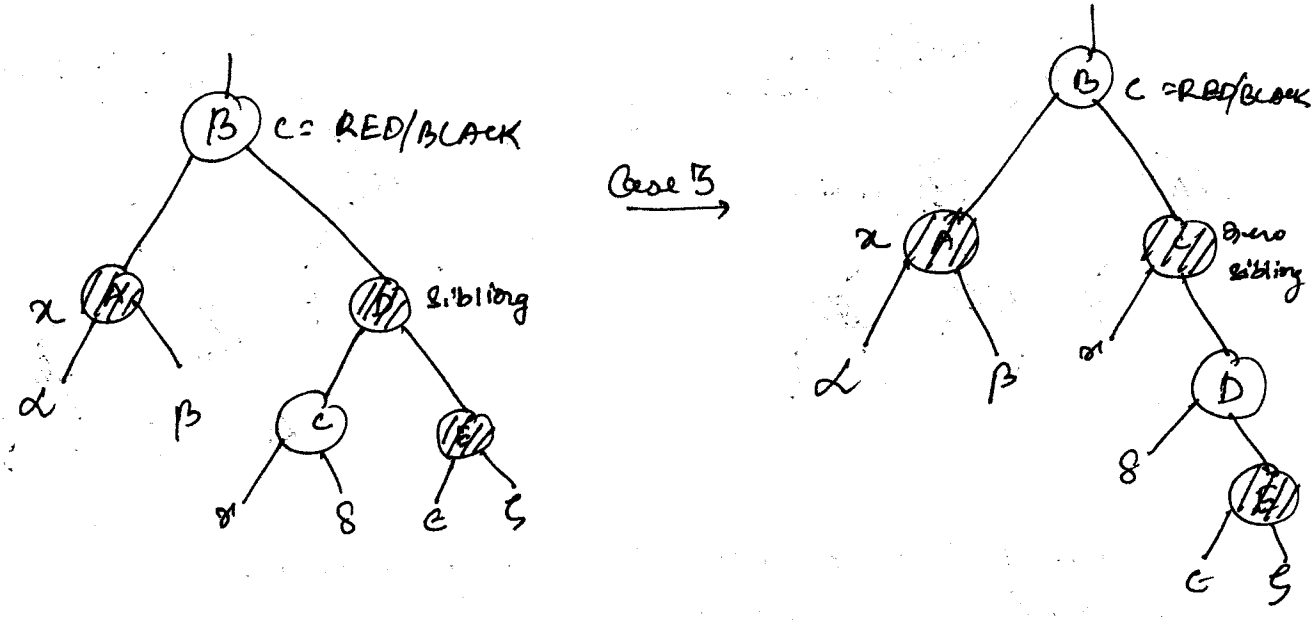


Conceptually, we remove one blackness from both x and its sibling. Since x was initially doubly black with its color attribute= BLACK, the color attribute of x remains so.

Since, the sibling was initially singly black, its color changes to red. Hence the execution of color[sibling] ← RED.

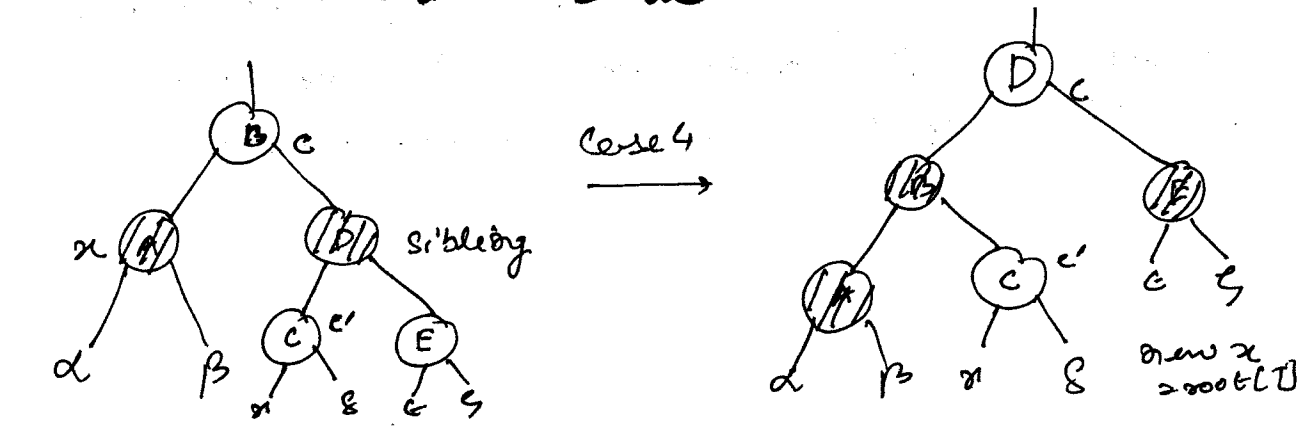
To keep the black counts constant we bestow an extra black on parent of x by letting x point to it. [Remember that any node pointed to by x has an extra blackness].

(iii) Case 3: Sibling is black. left child of sibling is red. right child of sibling is black.



We transform case 3 into case 4 by switching the colors of sibling and its left-child and then performing a right rotation on the sibling.

(iv) Case 4: Sibling is black, sibling's right child is red



sibling gets parents color. parent gets BLACK. left-rotate about parent. set x = root and terminate.

→ Run time of deletion = $O(\lg n)$. RB-DELETE take $O(\lg n)$ without fixup. Cases 1, 3 and 4 of fixup terminate after a constant no. of color changes and at max. 3 rotations. Case 2 may cause while loop to be repeated when the pointer x moves up the tree at most $O(\lg n)$ times. RB-DELETE-FIXUP takes $O(\lg n)$ time and performs at max. 3 rotations. Overall time for RB-DELETE is $O(\lg n)$.